DHAR

Bare Metal. Absolute Control.

VOLUME II — ENGINEERING ROADMAP

STAGE-ALPHA

"Forge of Dhar — a bridge built to be crossed once, then burned."

DOCUMENT	Stage-alpha — Forge of Dhar
VERSION	1.1
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume II
DEPENDS ON	Spec0, Spec1, Amendments A-1 and A-2
PRECEDES	Stage-beta — Honing Dhar

1. Mandate

Stage-alpha's job is singular and disposable: prove the full compilation pipeline once, end to end, for the smallest subset of Dhar capable of expressing Stage-beta's own source code, **and do so for every processor and operating system the Canon names, before the self-hosting compiler begins** (Amendment A-2). It is not the final compiler architecture, is not required to be fast, elegant, or complete beyond that subset, and is explicitly permitted — expected — to be retired once it has done this one job.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). Stage-alpha's part is the ground that rests on: a small compiler whose output is reproducible, that does only what it says, and that treats every target by the same test. No work on the three tracks begins before Stage-beta compiles itself (Amendment A-1 §7).

2. Scope

2.1 In scope

- Tokenizing `.dhar` source text.
- A recursive-descent parser (Spec0 §3, pillar 1) covering exactly the bootstrap subset defined in §2.2.
- A symbol table sufficient for the bootstrap subset's scoping rules.
- Direct assembly emission, with no intermediate representation, for **every committed target** (Amendment A-2 §3.1), and the system-interface stub each target needs (§3.2).
- The print function `gen` (Spec1 §20.2) and the system call `sys` (Spec1 §20.1).
- A conformance battery, and a runner for it, that is the same for every target (§3.3).
- File I/O sufficient to read source and write a native output binary.

2.2 The bootstrap subset

Stage-alpha **must** support, at minimum:

CATEGORY	REQUIRED CONSTRUCTS
Bindings	`lock`, `flux`
Functions	`task`, `give`
Branching	`when`, `shift`, `fallback`
Loops	`span`, `cycle`, `escape`, `skip`
Data	`mold` (layout only), array indexing, `i32`, `str`, `bool`, `raw[N]`
Modules & I/O	`pull`, `expose`, `gen`

CATEGORY	REQUIRED CONSTRUCTS
System	`sys` with the Linux x86-64 numbers (Spec1 §20.1), `sysret`

`forge`, the full keyword surface, collections beyond array indexing, `wild`, and every construct in Spec1 not listed above are explicitly **not required** to work in Stage-alpha, and Stage-alpha **must** reject them with a clear diagnostic rather than mishandling them silently. This includes the additions of Spec1 v3.2 (the wide and narrow numeric types, secret values, arenas): they are not part of the bootstrap subset, and a program that uses one is rejected with a diagnostic.

For `gen`, Stage-alpha accepts exactly the forms Spec1 §20.2 requires of every implementation: a string literal, a number literal, `true` or `false`, or the name of a variable of type `i32`, `str` or `bool`, whether its type was declared or inferred. A call, an indexed element or a field as the value is not part of the subset and is rejected with a diagnostic; the program binds it to a variable first.

The subset does not grow to serve a target. Where a target cannot express a construct that the subset allows, the compiler refuses that construct for that target with a diagnostic; it never emits a program that behaves differently (§3.1).

2.3 Out of scope

- The Universal IR and multi-backend architecture (Spec0 §4) — Stage-alpha emits directly and has no intermediate representation; the IR boundary, and the re-implementation of the back ends on it, begin at Stage-beta (Amendment A-2 §2, §7).
- Any target outside the committed list of Amendment A-2 §3.1. Such a target is refused (§3.1); it is never approximated by another target.
- Running the compiler on a host other than Linux x86-64 (Amendment A-2 §3.4); that follows from Stage-beta.
- Any optimization whatsoever.
- Self-hosting — Stage-alpha is, by definition, not written in Dhar.
- The work of the security, AI and quantum tracks (Amendment A-1 §7).

3. Required Architecture

Stage-alpha's source **must** be organized as one-definition-per-file modules from its first commit, not refactored into this shape later. Every label and data symbol **must** exist in exactly one file.

MODULE	RESPONSIBILITY
data.asm	Global data, keyword and string tables
entry.asm	Program entry point only — no other logic
tokenize.asm	Character scanning, token creation
parse.asm	Recursive-descent parsing per §2.2
symbols.asm	Symbol-table management

MODULE	RESPONSIBILITY
frames.asm	Stack-frame analysis
calls.asm	Function-call handling
codegen.asm	Code generation via the template-emitter approach (Spec0 §3, pillar 1)
targets.asm	The table of committed targets, and the check that refuses every other (§3.1)
prelude/*.asm	One file per committed target: its macros, entry sequence and system-interface stub (§3.1)
runtime.asm	Minimal runtime support, error reporting

The main entry file **must** consist only of `%include` directives and the program entry point — no routine may be defined there. This rule exists because splitting a monolith into modules only counts once nothing is left duplicated between the split files and the original: a module list is not modularity if the old file still contains everything it was split out of.

3.1 The neutral operation set and target preludes

Stage-alpha keeps its single token-walking emitter and has **no intermediate representation** (Amendment A-2 §4). What keeps each target small is a rule about the shape of the output:

- **A neutral operation set.** For the targets beyond the first, the emitter writes one neutral line per operation (load immediate, load, store, add, jump, call, and so on), with one operand slot per line, as the existing templates do.
- **One prelude per target.** Each target supplies a prelude of assembler macros that expand each neutral operation into that target's instructions, with its entry sequence and its system-interface stub. All of a target's knowledge sits in one file, `prelude/<target>.asm`, readable and checkable in one place. The output for a target is assembled and linked by that target's standard assembler and linker.
- **WebAssembly is structured.** It has a stack machine and structured control flow, so text templates that jump to labels do not translate directly. Its prelude uses a dispatcher form: one loop per task and a branch table over block numbers.
- **Ranges and alignment are the prelude's duty.** RISC-V conditional branches reach only a short distance, so each is emitted inverted over an unconditional jump; ARM64 requires 16-byte stack alignment at calls.
- **The neutral operation set and the first prelude come before any other target** (Amendment A-2 §7). The order of the targets is The Pact §2.1.
- **An unsupported target is a fatal error:** a diagnostic, a non-zero exit status, and no output file. The compiler never falls back to another target and never produces output for a target it has not been built and tested for (Amendment A-2 §2).
- **The host.** Stage-alpha is itself a program for Linux x86-64. It runs there and produces programs for every committed target; it is a cross-compiler.

The one list of committed targets is Amendment A-2 §3.1. This document does not repeat it.

3.2 The system interface and the print function

Programs call the system with `sys` and a Linux x86-64 number, or with the named forms (Spec1 §20.1). Each prelude translates the number to its target's interface: a raw system-call instruction on the Linux targets, the operating system's own library on Windows and macOS, the WASI imports on WebAssembly. So **one source program runs unchanged on every committed target** and does the same observable thing. A call a target cannot offer is refused at compile time for that target, or fails with a defined error at run time; it is never silently different.

`gen` (Spec1 §20.2) writes exactly the text of its value and nothing more on every target: a number in decimal, a boolean as `true` or `false`, a string as its bytes, no added newline, no trailing zero byte.

3.3 The conformance battery and the gate

The battery is one set of programs with the same expected results on every target:

1. **Positive programs**, whose output or checksum is fixed.
2. **The `gen` programs**, which must print exactly their text.
3. **The bounds-check programs**, which must stop with the defined failure: the text `Dhar: index out of bounds`, then a newline, on standard error (descriptor 2), and exit status 1 (Spec1 §14.4).
4. **The negative programs**, which must be rejected with a diagnostic and leave no output file.
5. **The refusal of unsupported targets:** every target outside Amendment A-2 §3.1 is refused as §3.1 says.

A target's gate has two levels (Amendment A-2 §6):

LEVEL	CONDITION	WHAT IT ALLOWS
GO, enabled	The whole battery passes when the target's output is run in an emulator or a runtime that enforces the target's rules	The compiler accepts the target on its command line; before this it refuses it
G1, complete	The whole battery passes when the target's output is run on the real operating system, on real hardware or in a real runtime for that target	The target counts toward Stage-alpha's exit criteria and may be described publicly as supported

An emulator is evidence and is recorded, but it does not pass G1. A WebAssembly runtime such as a WASI host is a real runtime for that target. A statement that Dhar supports a target names the gate and the run that passed it (Tempering Dhar §11).

4. Acceptance Criteria

1. Assembles and links into a native executable using only standard toolchain calls: for the host directly, and for every committed target with that target's standard assembler and linker.
2. Correctly tokenizes, parses, and compiles every construct in the bootstrap subset (§2.2), verified by at least one passing test per construct.
3. Rejects, with a clear diagnostic, any construct outside the bootstrap subset.
4. Successfully compiles Stage-beta's complete source tree end to end, producing a working Stage-beta executable.

5. Produces reproducible builds: two clean builds from the same source are byte-identical, ignoring timestamps; and the output for every committed target is byte-identical across two builds of the same program.
6. Every committed target has passed G1 (§3.3).
7. The conformance battery, the target tables of this document and Amendment A-2 §3.1 agree.
8. The compiler refuses every target outside Amendment A-2 §3.1 with a diagnostic, a non-zero exit status and no output file.
9. One source program of the battery produces the same observable behaviour on every committed target.

5. Handoff to Stage-beta

```
Stage-alpha compiles Stage-beta's source
         |
         v
Stage-beta executable (generation 1)
```

Stage-beta begins when Stage-alpha is complete on every committed target (§4, criterion 6); self-hosting work before that is preparation and does not count toward Stage-beta's gate (Amendment A-2 §2). Once this handoff succeeds and Stage-beta separately passes its own self-hosting acceptance criteria, Stage-alpha's mandate is complete. At the handoff Stage-alpha is **frozen and archived**, as Spec0 §3, pillar 5 says of the bridge: it **must not** be extended with new features, and any new capability belongs in Stage-beta or later, never retrofitted into the bridge. The archive stays reproducible, because the two-compiler check of Amendment A-1, S4 (Stage-beta §4) needs it. The output of its back ends remains the reference against which Stage-beta's back ends, re-implemented on the intermediate representation, are compared; each must pass the same battery (Amendment A-2 §7).

Revision log

Stage-alpha — Forge of Dhar · v1.1 · October 26, 2026 · Ratified.

VERSION	DATE	CHANGE
v1.0	September 6, 2026	First ratified edition.
v1.1	October 26, 2026	Amendment A-2 (Stage 0 targets): direct emission for every committed target, the neutral operation set and target preludes (§3.1), the system interface and gen on every target (§3.2), the conformance battery and the gates G0 and G1 (§3.3), exit criteria and a frozen hand-off (§4, §5). Amendment A-1: the direction (§1) and the order of the tracks (§5).